



北京航空航天大学计算机学院

School of Computer Science and Engineering, Beihang University

第十章 概率在语言模型的应用

李安南

2025年12月16日



提纲

- ① 基本概念
- ② n 元文法模型
- ③ n 元模型应用
- ④ n 元模型的问题
- ⑤ 语言模型的发展



基本概念

- 大规模**语料库**(Corpus)的出现为自然语言统计处理方法的实现提供了可能，统计方法的成功使用推动了**语料库语言学** (Corpus Linguistics) 的发展。
- 基于大规模语料库和统计方法，可以
 - 发现语言使用的普遍规律
 - 进行机器学习、自动获取语言知识
 - 对未知语言现象进行推测



- **语料库 (Corpus)** 是一个按照特定标准收集和组织的文本或语音数据的集合，通常用于语言学研究、自然语言处理 (NLP) 和机器学习等领域。它可以被看作是语言的“样本库”，为分析和理解语言现象提供实证基础。
- 语料库的核心特点：
 - **真实性**：语料库中的语言材料通常来自真实场景（如书籍、新闻、对话录音等），而非虚构例句。
 - **结构化**：数据经过整理、标注（如词性、句法、情感标签等），便于计算机或研究者系统分析。
 - **代表性**：根据研究目标设计，需反映特定语言变体（如学术英语、口语方言等）。
 - **规模可大可小**：从小型专用语料库（如某作家作品集）到超大型通用语料库（如数十亿词的互联网文本）。



语料的结构化

- 结构化就是为原始文本添加描述性的“标签”，使其从“一串字符”变成“可被检索和分析的信息网络”。

假设我们有一条原始文本（一条微博）

“这家新开的火锅店太赞了！强烈推荐毛肚和牛肉，服务也超贴心。” #美食推荐

第一层：文本元信息（描述语料自身的结构）

- 文本ID: T20231115001
- 来源: 新浪微博发布者: 用户A (可进一步标注性别、地区等, 如 性别=女, 地区=北京)
- 发布时间: 2023-11-15 19:30:00
- 体裁: 网络社交媒体帖子
- 主题/标签: #美食推荐
- 情感倾向 (整体): 正面



“这家新开的火锅店太赞了！强烈推荐毛肚和牛肉，服务也超贴心。” #美食推荐

第二层：文本形式结构（篇章结构）

- 文段落/句子切分：
 - 句子1：“这家新开的火锅店太赞了！”
 - 句子2：“强烈推荐毛肚和牛肉，服务也超贴心。”
- 标题/正文分离：这里没有明确标题，正文就是全部。



“这家新开的火锅店太赞了！强烈推荐毛肚和牛肉，服务也超贴心。” #美食推荐

第三层：语言学标注（最核心的结构化）

(1) 分词与词性标注

将句子切成独立的词或语素，并标注其词性（名词、动词等）。

➤ 这/r 家/q 新/a 开/v 的/u 火锅店/n 太/d 赞/a 了/y ! /w

➤ r=代词， q=量词， a=形容词， v=动词， u=助词， n=名词， d=副词， y=语气词， w=标点

作用：这是几乎所有NLP任务的基础。可以统计词频（“火锅店”出现了多少次）、分析词性分布（美食评论中形容词的使用比例）等。



“这家新开的火锅店太赞了！强烈推荐毛肚和牛肉，服务也超贴心。” #美食推荐

第三层：语言学标注（最核心的结构化）

(2) 句法分析（短语/依存结构）

短语结构： [NP [DP 这 家] [AP 新 开 的] 火锅店] [VP 太 赞 了] （这是一个名词短语+动词短语的主谓结构）

依存关系（更常见）：

赞（核心谓语）

→ 主语：火锅店

→ 程度修饰：太

→ 时态：了

火锅店

→ 限定词：这

→ 量词：家

→ 定语：开（“开”又依赖于“新”作状语）



“这家新开的火锅店太赞了！强烈推荐毛肚和牛肉，服务也超贴心。” #美食推荐

第三层：语言学标注（最核心的结构化）

(3) 命名实体识别

识别文本中具有特定意义的实体。

- 这家新开的[ORG 火锅店]太赞了！强烈推荐[FOOD 毛肚]和[FOOD 牛肉]，[ABSTRACT 服务]也超贴心。
- ORG=组织机构， FOOD=食物， ABSTRACT=抽象概念



“这家新开的火锅店太赞了！强烈推荐毛肚和牛肉，服务也超贴心。” #美食推荐

第三层：语言学标注（最核心的结构化）

（4）语义角色标注

标注句子中“谁对谁做了什么”，即谓词-论元关系

- **谓词:** 推荐
 - 推荐者（施事）：（隐含的“我”）
 - 推荐内容（受事）：毛肚和牛肉
- **谓词:** 贴心
 - 属性主体（系事）：服务
 - 程度: 超



“这家新开的火锅店太赞了！强烈推荐毛肚和牛肉，服务也超贴心。” #美食推荐

第四层：应用导向的标注

情感/评价标注（细粒度）：

- [目标：火锅店， 评价词：赞， 极性：正面， 强度：高]
- [目标：毛肚， 评价词：推荐， 极性：正面]
- [目标：牛肉， 评价词：推荐， 极性：正面]
- [目标：服务， 评价词：贴心， 极性：正面， 强度：高]



如何计算一段文字(句子)的概率?

《概率论与数理统计》是高等院校理工类、经管类的重要课程之一。在考研数学中的比重大约占22%左右（数一、数三）。

- 以一段文字(句子)为单位统计相对频率?
- 根据句子构成单位的概率计算联合概率?

$$p(w_1) \times p(w_2) \times \dots \times p(w_n)$$



语句 $s = w_1 w_2 \dots w_m$ 的先验概率:

$$\begin{aligned} p(s) &= p(w_1) \times p(w_2/w_1) \times p(w_3/w_1w_2) \times \dots \\ &\quad \times p(w_m/w_1 \dots w_{m-1}) \\ &= \prod_{i=1}^m p(w_i \mid w_1 \dots w_{i-1}) \end{aligned}$$

当 $i=1$ 时, $p(w_1|w_0) = p(w_1)$ 。



说明:

- w_i 可以是**字、词、短语或词类**等等，称为统计**基元**。通常以“词”代之。
- w_i 的概率由 $w_1, \dots, w_{i-1}$ 决定，由特定的一组 $w_1, \dots, w_{i-1}$ 构成的一个序列，称为 w_i 的**历史**。



问题

- 随着历史基元的增加，不同的“历史”（路径）按**指数级**增长。
- 对于第 i ($i > 1$) 个统计基元，历史基元的个数为 $i-1$ ，如果共有 L 个不同的基元（如词汇表），理论上每个单词都可能出现现在1到 $i-1$ 的每个位置上，那么， i 基元就有 L^{i-1} 种不同的历史。
- 我们必须考虑在所有的 L^{i-1} 种不同历史下产生第 i 个基元的概率。那么，模型中有 L^m 个参数 $p(w_m | w_1 \dots w_{m-1})$ 。



如何解决？

设法减少历史基元的个数，将 $w_1 w_2 \dots w_{i-1}$ 映射到等价类 $S(w_1 w_2 \dots w_{i-1})$ ，使等价类的数目远远小于原来不同历史基元的数目。则有：

$$p(w_i \mid w_1, \dots, w_{i-1}) = p(w_i \mid S(w_1, \dots, w_{i-1}))$$



如何划分等价类?

将两个历史映射到同一个等价类, 当且仅当这两个历史中的最近 $n-1$ 个基元相同, 即:

$$\begin{array}{ccccccc} H_1: & w_1 & w_2 & \dots & \dots & \underbrace{w_{i-n+1} w_{i-n+2} \dots w_{i-1}}_{n-1} & w_i \dots \dots \\ & & & & & & \uparrow \\ & & & & & & \downarrow \\ H_2: & v_1 & v_2 & \dots & \dots & \underbrace{v_{k-n+1} v_{k-n+2} \dots v_{k-1}}_{n-1} & v_k \dots \dots \end{array}$$

$$S(w_1, w_2, \dots, w_i) = S(v_1, v_2, \dots, v_k)$$

$$\text{iff } H_1: (w_{i-n+1}, \dots, w_i) = H_2: (v_{k-n+1}, \dots, v_k)$$



提纲

- ① 基本概念
- ② n 元文法模型
- ③ n 元模型应用
- ④ n 元模型的问题
- ⑤ 语言模型的发展



n 元文法模型

这种情况下的语言模型称为 n 元文法(n -gram)模型。

通常地,

- ❖ 当 $n=1$ 时, 即出现在第 i 位上的基元 w_i 独立于历史。
一元文法也被写为 uni-gram 或 monogram;
- ❖ 当 $n=2$ 时, 2-gram (bi-gram) 被称为1阶马尔可夫链;
- ❖ 当 $n=3$ 时, 3-gram(tri-gram)被称为2阶马尔可夫链,
依次类推。



n 元文法模型

为了保证条件概率在 $i=1$ 时有意义，同时为了保证句子内所有字符串的概率和为 1，即 $\sum_s p(s) = 1$ ，可以在句子首尾两端增加两个标志: **<BOS>** $w_1 w_2 \dots w_m$ **<EOS>**。不失一般性，对于 $n>2$ 的 n -gram, $p(s)$ 可以分解为:

$$p(s) = \prod_{i=1}^{m+1} p(w_i \mid w_{i-n+1}^{i-1})$$

其中, w_i^j 表示词序列 $w_i \dots w_j$, w_{i-n+1} 从 w_0 开始, w_0 为 **<BOS>**, w_{m+1} 为 **<EOS>**。



n 元文法模型 举例

给定句子: John read a book

增加标记: **<BOS>** John read a book **<EOS>**

Unigram: <BOS>, John, read, a, book, <EOS>

Bigram: (<BOS>John), (John read), (read a),
(a book), (book <EOS>)

Trigram: (<BOS>John read), (John read a),
(read a book), (a book <EOS>)



n 元语法模型 举例

<BOS> John read a book <EOS>

基于2元语法的概率为:

$$\begin{aligned} p(\text{John read a book}) &= p(\text{John} | \text{<BOS>}) \times \\ &\quad p(\text{read} | \text{John}) \times p(\text{a} | \text{read}) \times \\ &\quad p(\text{book} | \text{a}) \times p(\text{<EOS>} | \text{book}) \end{aligned}$$



n 元语法模型 实现

创建一个N-gram模型，该模型可接受文本字符串，并将其分割成n-gram序列。

```
1  def create_ngram_model(text, n):
2      words = text.split() # 将文本分割成单词
3      ngrams = [] # 用于存储n-grams的列表
4
5      for i in range(len(words) - n + 1):
6          ngram = ' '.join(words[i:i + n]) # 创建一个n-gram
7          ngrams.append(ngram)
8
9      return ngrams
10
11 n = 2 # 选择2-gram模型
12 ngram_model = create_ngram_model(text, n)
13
14 # 打印前10个2-grams
15 print(ngram_model[:10])
```

create_ngram_model函数以文本text和n为参数，返回n-gram的列表。选择了2-gram模型（bigram），并打印前10个。



◆应用-1：音字转换问题

给定拼音串： ta shi yan jiu sheng wu de

可能的汉字串： 踏实研究生物的

他实验救生物的

他使烟酒生物的

他是研究生物的

... ..



$$\hat{CString} = \arg \max_{CString} p(CString | Pinyin)$$

$$= \arg \max_{CString} \frac{p(Pinyin | CString) \times p(CString)}{p(Pinyin)}$$

$$= \arg \max_{CString} p(Pinyin | CString) \times p(CString)$$

$$= \arg \max_{CString} p(CString)$$



$CString = \{\text{踏实研究生物的, 他实验救生物的, 他是研究生物的, 他使烟酒生雾的,}\}$

如果使用 2-gram:

$$p(CString_1) = p(\text{踏实}|\text{<BOS>}) \times p(\text{研究}|\text{踏实}) \times \\ p(\text{生物}|\text{研究}) \times p(\text{的}|\text{生物}) \times p(\text{<EOS>}|\text{的})$$

$$p(CString_2) = p(\text{他}|\text{<BOS>}) \times p(\text{实验}|\text{他}) \times p(\text{救}|\text{实验}) \times \\ p(\text{生物}|\text{救}) \times p(\text{的}|\text{生物}) \times p(\text{<EOS>}|\text{的})$$

.....



如果汉字的总数为： N

- 一元语法：
 - 1) 样本空间为 N
 - 2) 只选择使用频率最高的汉字
- 2元语法：
 - 1) 样本空间为 N^2
 - 2) 效果比一元语法明显提高
- 估计对汉字而言四元语法效果会好一些
- 智能狂拼、微软拼音输入法基于 n -gram.



◆应用-2：汉语分词问题

给定汉字串：他是研究生物的。

可能的汉字串：

- 1) 他|是|研究生|物|的
- 2) 他|是|研究|生物|的



$$\hat{Seg} = \arg \max_{Seg} p(Seg | Text)$$

$$= \arg \max_{Seg} \frac{p(Text | Seg) \times p(Seg)}{p(Text)}$$

$$= \arg \max_{Seg} p(Text | Seg) \times p(Seg)$$

$$= \arg \max_{Seg} p(Seg)$$



n 元语法模型 应用

如果采用2元文法：

$$p(\text{Seg1}) = p(\text{他}|\text{<BOS>}) \times p(\text{是}|\text{他}) \times p(\text{研究生}|\text{是}) \times \\ p(\text{物}|\text{研究生}) \times p(\text{的}|\text{物}) \times p(\text{的}|\text{<EOS>})$$

$$p(\text{Seg2}) = p(\text{他}|\text{<BOS>}) \times p(\text{是}|\text{他}) \times p(\text{研究}|\text{是}) \times \\ p(\text{生物}|\text{研究}) \times p(\text{的}|\text{生物}) \times \\ p(\text{的}|\text{<EOS>})$$

问题： 如何获得 n 元语法模型？



◆两个重要概念：

- 训练语料(training data)：用于建立模型，确定模型参数的已知语料。
- 最大似然估计(maximum likelihood Evaluation, MLE)：用相对频率计算概率的方法。



对于 n -gram, 参数 $p(w_i | w_{i-n+1}^{i-1})$ 可由最大似然估计求得:

$$p(w_i | w_{i-n+1}^{i-1}) = f(w_i | w_{i-n+1}^{i-1}) = \frac{c(w_{i-n+1}^i)}{\sum_{w_i} c(w_{i-n+1}^i)}$$

其中, $\sum_{w_i} c(w_{i-n+1}^i)$ 是历史串 w_{i-n+1}^{i-1} 在给定语料中出现的次数, 即 $c(w_{i-n+1}^{i-1})$, 不管 w_i 是什么。

$f(w_i | w_{i-n+1}^{i-1})$ 是在给定 w_{i-n+1}^{i-1} 的条件下 w_i 出现的相对频度, 分子为 w_{i-n+1}^{i-1} 与 w_i 同现的次数。



例如，给定训练语料：

“John read Moby Dick”,

“Mary read a different book”,

“She read a book by Cher”

根据 2 元文法求句子的概率？



参数估计

$$p(\text{John} | \langle \text{BOS} \rangle) = \frac{c(\langle \text{BOS} \rangle \text{John})}{\sum_w c(\langle \text{BOS} \rangle w)} = \frac{1}{3} \quad p(a | \text{read}) = \frac{c(\text{read } a)}{\sum_w c(\text{read } w)} = \frac{2}{3}$$

$$p(\text{read} | \text{John}) = \frac{c(\text{John } \text{read})}{\sum_w c(\text{John } w)} = \frac{1}{1} \quad p(\text{book} | a) = \frac{c(a \text{ book})}{\sum_w c(a w)} = \frac{1}{2}$$

$$p(\langle \text{EOS} \rangle | \text{book}) = \frac{c(\text{book } \langle \text{EOS} \rangle)}{\sum_w c(\text{book } w)} = \frac{1}{2}$$

$$p(\text{John read a book}) = \frac{1}{3} \times 1 \times \frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} \approx 0.06$$

$\langle \text{BOS} \rangle \text{John read Moby Dick} \langle \text{EOS} \rangle$

$\langle \text{BOS} \rangle \text{Mary read a different book} \langle \text{EOS} \rangle$

$\langle \text{BOS} \rangle \text{She read a book by Cher} \langle \text{EOS} \rangle$



参数估计

$$p(\textit{Cher read a book}) = ?$$

$$= p(\textit{Cher} | \langle \textit{BOS} \rangle) \times p(\textit{read} | \textit{Cher}) \times p(\textit{a} | \textit{read}) \times \\ p(\textit{book} | \textit{a}) \times p(\langle \textit{EOS} \rangle | \textit{book})$$

$$p(\textit{Cher} | \langle \textit{BOS} \rangle) = \frac{c(\langle \textit{BOS} \rangle \textit{Cher})}{\sum_w c(\langle \textit{BOS} \rangle w)} = \frac{0}{3}$$

$$p(\textit{read} | \textit{Cher}) = \frac{c(\textit{Cher} \textit{read})}{\sum_w c(\textit{Cher} w)} = \frac{0}{1}$$



于是, $p(\textit{Cher read a book}) = 0$

$\langle \textit{BOS} \rangle \textit{John read Moby Dick} \langle \textit{EOS} \rangle$

$\langle \textit{BOS} \rangle \textit{Mary read a different book} \langle \textit{EOS} \rangle$

$\langle \textit{BOS} \rangle \textit{She read a book by Cher} \langle \textit{EOS} \rangle$



问题:

数据匮乏(稀疏) (*Sparse Data*) 引起零概率问题,
如何解决?

数据平滑(data smoothing)



◆数据平滑的基本思想：

调整最大似然估计的概率值, 使零概率增值, 使非零概率下调, “劫富济贫”, 消除零概率, 改进模型的整体正确率。

◆基本目标：测试样本的语言模型困惑度越小越好。

◆基本约束： $$\sum_{w_i} p(w_i | w_1, w_2, \dots, w_{i-1}) = 1$$



➤ 回顾一困惑度的定义：

对于一个平滑的 n -gram，其概率为 $p(w_i | w_{i-n+1}^{i-1})$ ，

可以计算句子的概率：

$$p(s) = \prod_{i=1}^{m+1} p(w_i | w_{i-n+1}^{i-1})$$

假定测试语料 T 由 l_T 个句子构成 $(t_1, \dots, t_{l_T})$ ，则整个测试集的概率为：

$$p(T) = \prod_{i=1}^{l_T} p(t_i)$$



◆ 数据平滑方法

加1法(Additive smoothing)

基本思想：每一种情况出现的次数加1。

例如，对于 *uni-gram*，设 w_1, w_2, w_3 三个词，概率分别为：1/3, 0, 2/3，加1后情况？

2/6, 1/6, 3/6



对于2-gram 有:

$$\begin{aligned} p(w_i | w_{i-1}) &= \frac{1 + c(w_{i-1}w_i)}{\sum_{w_i} [1 + c(w_{i-1}w_i)]} \\ &= \frac{1 + c(w_{i-1}w_i)}{|V| + \sum_{w_i} c(w_{i-1}w_i)} \end{aligned}$$

其中， V 为被考虑语料的词汇量(全部可能的基元数)。



数据平滑

在前面 3 个句子的例子中，

$$p(\textit{Cher read a book}) = p(\textit{Cher}|\langle\textit{BOS}\rangle) \times \\ p(\textit{read}|\textit{Cher}) \times p(\textit{a}|\textit{read}) \times p(\textit{book}|\textit{a}) \times \\ p(\langle\textit{EOS}\rangle|\textit{book})$$

$\langle\textit{BOS}\rangle\textit{John read Moby Dick}\langle\textit{EOS}\rangle$
 $\langle\textit{BOS}\rangle\textit{Mary read a different book}\langle\textit{EOS}\rangle$
 $\langle\textit{BOS}\rangle\textit{She read a book by Cher}\langle\textit{EOS}\rangle$

原来:

$$p(\textit{Cher}|\langle\textit{BOS}\rangle) = 0/3$$

$$p(\textit{read}|\textit{Cher}) = 0/1$$

$$p(\textit{a}|\textit{read}) = 2/3$$

$$p(\textit{book}|\textit{a}) = 1/2$$

$$p(\langle\textit{EOS}\rangle|\textit{book}) = 1/2$$



数据平滑

词汇量: $|V|=11$

平滑以后:

<BOS>John read Moby Dick<EOS>

<BOS>Mary read a different book<EOS>

<BOS>She read a book by Cher<EOS>

$$p(\text{Cher}|\text{<BOS>}) = (0+1)/(11+3) = 1/14$$

$$p(\text{read}|\text{Cher}) = (0+1)/(11+1) = 1/12$$

$$p(\text{a}|\text{read}) = (1+2)/(11+3) = 3/14$$

$$p(\text{book}|\text{a}) = (1+1)/(11+2) = 2/13$$

$$p(\text{<EOS>}|\text{book}) = (1+1)/(11+2) = 2/13$$

$$p(\text{Cher read a book}) = \frac{1}{14} \times \frac{1}{12} \times \frac{3}{14} \times \frac{2}{13} \times \frac{2}{13} \approx 0.00003$$



数据平滑

同理，对于句子 *John read a book* 数据平滑后：

$$p(\text{John}|\langle \text{BOS} \rangle) = 2/14, \quad p(\text{read}|\text{John}) = 2/12,$$

$$p(\text{a}|\text{read}) = 3/14, \quad p(\text{book}|\text{a}) = 2/13, \quad p(\langle \text{EOS} \rangle|\text{book}) = 2/13$$

$$\text{于是, } p(\text{John read a book}) = p(\text{John}|\langle \text{BOS} \rangle) \times p(\text{read}|\text{John}) \times \\ p(\text{a}|\text{read}) \times p(\text{book}|\text{a}) \times p(\langle \text{EOS} \rangle|\text{book})$$

$$= \frac{2}{14} \times \frac{2}{12} \times \frac{3}{14} \times \frac{2}{13} \times \frac{2}{13} \approx 0.0001$$

$\langle \text{BOS} \rangle$ John read Moby Dick $\langle \text{EOS} \rangle$

$\langle \text{BOS} \rangle$ Mary read a different book $\langle \text{EOS} \rangle$

$\langle \text{BOS} \rangle$ She read a book by Cher $\langle \text{EOS} \rangle$



提纲

1

基本概念

2

n 元文法模型

3

n 元模型应用

4

n 元模型的问题

5

语言模型的发展



◆ 汉语分词问题：

句子：这篇文章写得太平淡了。

这 / 篇 / 文章 / 写 / 得 / 太 / 平淡 / 了 / 。

这 / 篇 / 文章 / 写 / 得 / 太平 / 淡 / 了 / 。



采用基于语言模型的分词方法

➤ 方法描述:

设对于待切分的句子 $S = z_1 z_2 \dots z_m$, $W = w_1 w_2 \dots w_k$ ($1 \leq k \leq n$) 是一种可能的切分。那么,

$$\begin{aligned}\hat{W} &= \arg \max_W p(W | S) \\ &= \arg \max_W p(W) \times p(S | W) \\ &\cong \arg \max_W p(W)\end{aligned}$$

最基本的做法是以词为独立的统计基元，但效果不佳。



具体实现时，可把词汇分成如下几类：

- (1) 分词词典中规定的词;
- (2) 由词法规则派生出来的词或短语，如：干干净净、非党员、全面性、检查员、看不出、克服了、走出来、洗个澡 ...
- (3) 与数字相关的实体，如：日期、时间、货币、百分数、温度、长度、面积、重量、电话号码、邮件地址等;
- (4) 专用名词，如：人名、地名、机构名。



进一步做如下约定，把一个可能的词序列 W 转换成词类序列 $C = c_1 c_2 \cdots c_N$ ，即：

- 专有名词：人名**PN**、地名**LN**、机构名**ON**分别作为一类；
- 实体名词中的日期 **dat**、时间**tim**、百分数**per**、货币**mon** 等作为一类；
- 对词法派生词**MW**和词表词**LW**，每个词单独作为一类。



例如：

3月14日下午3点比尔盖茨在北京发表讲话，决定从今年起微软亚洲研究院将大规模招收研发人员，其中，80%将从中国科学院大学所培养的应届博士或硕士毕业生中选拔，年薪不低于3万美元。



n元模型应用举例

日期dat

时间tim

人名PN

地名LN

3月14日下午3点比尔盖茨在北京发表讲话，决定从
今年起微软亚洲研究院将大规模招收研发人员，其
中，80%将从中国科学院大学所培养的应届博士或
硕士毕业生中选拔，年薪不低于3万美元。

机构名ON

百分数per

货币数mon



词序列变为类序列：

dat/ tim/ PN/ 在/ LN/ 发表/ 讲话/ ， / 决定/ 从/
tim/ 起/ ON/ 将/ 大/ 规模/ 招收/ 研发/ 人员/ ， /
其中/ ， / per/ 将/ 从/ 中国/ 科学院/ 大学/ 所/ 培
养/ 的/ 应届/ 博士/ 或/ 硕士/ 毕业生/ 中/ 选拔/ ， /
年薪/ 不/ 低于/ mon/ 。



n元模型应用举例

那么, $\hat{C} = \arg \max_C p(C | S)$

$$= \arg \max_C p(C) \times p(S | C) \dots (1)$$

语言模型

生成模型

$p(C)$ 可采用三元语法:

$$p(C) = p(c_1) \times p(c_2 | c_1) \prod_{i=3}^N p(c_i | c_{i-2}c_{i-1}) \quad (2)$$

$$p(c_i | c_{i-2}c_{i-1}) = \frac{\text{count}(c_{i-2}c_{i-1}c_i)}{\text{count}(c_{i-2}c_{i-1})} \quad (3)$$



n元模型应用举例

$$\begin{aligned}\hat{Seg} &= \arg \max_{Seg} p(Seg | Text) \\ &= \arg \max_{Seg} \frac{p(Text | Seg) \times p(Seg)}{p(Text)} \\ &= \arg \max_{Seg} p(Text | Seg) \times p(Seg) \\ &= \arg \max_{Seg} p(Seg)\end{aligned}$$

不考虑这部分的理由只是为了简化和近似，并没有严格的数学依据。当然也可以不省简。



生成模型在满足独立性假设的条件下，可近似为：

$$p(S | C) \approx \prod_{i=1}^N p(s_i | c_i) \quad (4)$$

该公式的含意是，任意一个词类 c_i 生成汉字串 s_i 的概率只与自身有关，而与其上下文无关。例如，如果“教授”是词表里的词，那么

$p(s_i=\text{教授} | c_i=\text{LW})=1$ ，否则， $p(s_i | c_i) = 0$ 。



n元模型应用举例

词 类	生成模型 $p(S C)$	语言知识来源
词表词 (LW)	若S是词表词, $p(S LW)=1$, 否则为0;	分词词表
词法派生词 (MW)	若S是派生词, $p(S MW)=1$, 否则为0;	
人名 (PN)	基于字的2元模型	
地名 (LN)	基于字的2元模型	地名简称表
机构名 (ON)	基于词类的2元模型	机关名关键词表, 机构名简称表
其他实体名 (FT)	若S可用实体名词规则集G识别, $p(S G)=1$, 否则为0。	实体名词规则集

$$p(S | C) \approx \prod_{i=1}^N p(s_i | c_i)$$

?



n元模型应用举例

词 类	生成模型 $p(S C)$	语言知识来源
词表词 (LW)	若S是词表词, $p(S LW)=1$, 否则, 判断是否其它类别;	分词词表
词法派生词 (MW)	若S是派生词, $p(S MW)=1$, 否则, 判断是否其它类别;	派生词词表
人名 (PN)	基于字的2元模型	姓氏表, 中文人名模板
地名 (LN)	基于字的2元模型	地名表、地名关键词表、 地名简称表
机构名 (ON)	基于词类的2元模型	机关名关键词表, 机构 名简称表
其他实体名 (FT)	若S可用实体名词规则集G识别, $p(S G)=1$, 否则, 判断是否其它类别。	实体名词规则集



n元模型应用举例

模型的训练由以下几步组成:

- (1) 在词表和派生词表的基础上, 用基本的分词工具切分训练语料, 专有名词通过专门模块标注, 实体名词通过相应的规则和有限状态自动机标注, 由此产生一个带词类别标记的初始语料;
- (2) 用带词类别标记的初始语料, 采用最大似然估计方法估计语言模型的概率参数, 公式(3);
- (3) 用得到的模型(公式(1)、(2)、(4))对训练语料重新切分和标注, 得到新的训练语料;
- (4) 重复(2)(3)步, 直到系统的性能不再有明显的变化为止。



n元模型应用举例

实验：

- (1)词表词：98,668条、派生词：59,285条；
- (2)训练语料：88MB 新闻文本；
- (3)测试集：247,039个词次，分别来自描写文、叙述文、说明文、口语等。

测试指标：

$$\text{正确率} = \frac{\text{切分正确的词数}}{\text{系统输出的总词数}} \times 100\% = 96.3\%$$

黄昌宁，高剑峰，李沐，对自动分词的反思，见：2003年全国第七届计算语言学联合学术会议论文集，pp.26-38



◆分词与词性标注一体化方法

汉语分词：这篇文章写得太平淡了。

这/ 篇/ 文章/ 写/ 得/ 太/ 平淡/ 了/ 。

这/ 篇/ 文章/ 写/ 得/ 太平/ 淡/ 了/ 。

标注词性后：这/**P** 篇/**M** 文章/**N** 写/**V** 得/**D** 太/**D** 平淡/**A**
了/**X** 。/**B**



假设句子 S 是由单词串组成:

$$W = w_1 w_2 \cdots w_n \quad (n \geq 1)$$

单词 w_i ($1 \leq i \leq n$) 的词性标注为 t_i , 即句子 S 相应的词性标注符号序列可表达为:

$$T = t_1 t_2 \cdots t_n$$

那么, 分词与词性标注的任务就是要在 S 所对应的各种切分和标注形式中, 寻找 T 和 W 的联合概率 $p(W, T)$ 为最优的词切分和标注组合。



(1)基于词性的三元统计模型：

$$p(W, T) = p(W | T) \times p(T) \approx \prod_{i=1}^n p(w_i | t_i) \times p(t_i | t_{i-1}, t_{i-2})$$

其中， $p(W|T)$ 称为生成模型， $p(w_i | t_i)$ 表示在整个标注语料中，在词性 t_i 的条件下，单词 w_i 出现的概率。

$p(T)$ 为基于词性的语言模型，采用三元文法，

当 $i=1$ 时，取 $p(t_1)$ ；当 $i=2$ 时，取 $p(t_2|t_1)$ 。



(2)基于单词的三元统计模型：

$$p(W, T) = p(T | W) \times p(W) \approx \prod_{i=1}^n p(t_i | w_i) \times p(w_i | w_{i-1}, w_{i-2})$$

其中， $p(t_i | w_i)$ 反映的是每个词对应词性符号的概率。
 $p(w_i | w_{i-1}, w_{i-2})$ 是普通的三元语言模型，当 $i=1$ 时，取 $p(w_1)$ ；当 $i=2$ 时，取 $p(w_2 | w_1)$ 。



(3)分词与词性标注一体化模型：

$$p^*(W, T) = \alpha \prod_{i=1}^n p(w_i | t_i) \times p(t_i | t_{i-1}, t_{i-2}) + \beta \prod_{i=1}^n p(t_i | w_i) \times p(w_i | w_{i-1}, w_{i-2})$$

这种综合模型的指导思想是希望通过调整参数alpha 和beta的值来确定两个子模型在整个分词与词性标注过程中所发挥作用的比重，从而获得分词与词性标注的整体最优。（当i 分别为1和2时的语言模型分别取一元文法概率和二元文法概率。）



说明:

- 对于汉语分词而言，不同的分词方法往往各有千秋，不要简单地从正确率高低上判断方法的好坏，正确率只从某一侧面反映了方法的性能；
- 除了汉语自动分词以外，语言模型广泛地应用于自然语言处理的各个方面，是统计自然语言处理方法中最核心、最基本的模型。



n元模型分词的实现

```
1  例句S: 我是北京大学的一名研究生
2  # S的切分可以拆成两步
3   $P(S) = P(\text{我}) * P(\text{是北京大学的一名研究生})$ 
4  #同时后面的子句, 继续可以拆成:
5   $P(S) = P(\text{我}) * P(\text{是北京大学的一名研究生}) = P(\text{我}) * P(\text{是}) * p(\text{北京大学的一名研究生})$ 
6  #这里有个问题, 我们是如何知道应该拆成“我”和“是”两个词, 而不是“我是”一个词呢
7  #上面计算最大概率, 是递归调用的, 假设我们开始有两种切分
8  [我, 是北京大学的一名研究生]
9  [我是, 北京大学的一名研究生]
10 #计算组合概率
11  $P1 = P(\text{我}) * P(\text{是北京大学的一名研究生})$ 
12  $P2 = P(\text{我是}) * P(\text{北京大学的一名研究生})$ 
13 #我们会发现
14  $P1 > P2$ 
15 #对于后面的任何子句, 我们都采用无脑切分, 即设置最大词长, 这里假设为3, 可得到以下切分:
16 [我, 是北京大学的一名研究生]
17 [我是, 北京大学的一名研究生]
18 [我是北, 京大学的一名研究生]
19 #然后分别递归计算
20 #为了满足性能的要求, 避免重复计算, 我们采用将间接计算的子序列的组合概率, 都存储起来
21 #每次计算新的子序列时, 先查看子序列的切分组合中, 是否包含已经计算过的子序列, 包含, 则直接复用
```



n元模型分词的实现

```
1 def maxP(self, sentence):
2     '''
3     计算最大切分方案
4     :param sentence: 待切分句子
5     :return:
6     '''
7     # 遍历所有切分组合中, 找出最大概率切分
8     if len(sentence) <= 1:
9         return self.DICT.getPValue(self, sentence)
10    # 判断切词方向: backward 或 forward
11    sentence_split_words = [self.backwardSplitSentence(
12        sentence), self.forwardSplitSentence(sentence)][self.split_way != 'back']
13    # 记录最大概率值
14    max_p_value = 0
15    # 储存最大概率下的切分组合
16    word_pairs = []
17    # 组合概率值
18    word_p = 0
19    for pair in sentence_split_words:
20        p1, p2 = 0, 0
21        if pair[0] in self.value_dict:
22            p1 = self.value_dict[pair[0]]
23        else:
24            p1 = self.maxP(pair[0])
25        if pair[1] in self.value_dict:
26            p2 = self.value_dict[pair[1]]
27        else:
28            p2 = self.maxP(pair[1])
29        word_p = p1 * p2
30        if max_p_value < word_p:
31            max_p_value = word_p
32            word_pairs = pair
33    # 在词典中查询当前句对应的频率, 不存在时, 返回 1/N
34    sentence_p_value = self.DICT.getPValue(self, sentence)
35    # 不切分概率最大时, 更新各值
36    if sentence_p_value > max_p_value and self.DICT.inDict(self, sentence):
37        self.value_dict[sentence] = sentence_p_value
38        self.seg_dict[sentence] = sentence
39        return sentence_p_value
40    # 某种切分组合概率最大时, 更新sentence对应概率, 避免后续切分重复计算
41    else:
42        self.value_dict[sentence] = max_p_value
43        self.seg_dict[sentence] = word_pairs
44        return max_p_value
```

Unigram切词基本思路:
首先统计出训练集词典中各个词的频率, 用来表示公式中的 w_i ;
然后, 对待切分文本的进行切分, 递归选择切分概率最大的子切分序列, 最后回溯得到最大概率切分。

平滑, 对于词典中搜索不到的词, 需要做一定的平滑处理。

建议动手尝试基于二元/三元模型的分词、词性标注等。



提纲

1

基本概念

2

n 元文法模型

3

n元模型应用

4

n元模型的问题

5

语言模型的发展



n元模型的问题

该课程很枯燥，大家觉得很无聊。

$$P(\text{无聊}|\text{很}) \\ = \frac{\text{count}(\text{很 无聊})}{\text{count}(\text{很})}$$

问题①：数据稀疏
N-元组“很 无聊”未出现过，则回退

问题②：忽略语义相似性
“无聊”与“枯燥”虽语义相似，但无法共享信息



n元模型的问题

- 典型方法：抽象符号（字符串）

该课程很枯燥，大家觉得很无聊。

w_0 =该 w_1 =课程 w_2 =很 w_3 =枯燥 w_4 =，
 w_5 =大家 w_6 =觉得 w_7 =很 w_8 =无聊 w_9 =。

- 等价表示方法：one-hot表示法

$|V|$

$\begin{bmatrix} \vdots \end{bmatrix}$



所有词按照出现的顺序排序



每个词语将对应唯一的下标

枯燥

$\begin{bmatrix} 0 \\ \mathbf{1} \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$

无聊

$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \mathbf{1} \\ 0 \end{bmatrix}$



n元模型的问题

- 问题

枯燥

$$\begin{bmatrix} 0 \\ \mathbf{1} \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

无聊

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \mathbf{1} \\ 0 \end{bmatrix}$$

枯燥 $\otimes$ 无聊

$$\begin{bmatrix} 0 \\ \mathbf{1} \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$\times$

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \mathbf{1} \\ 0 \end{bmatrix}$$

$= 0$

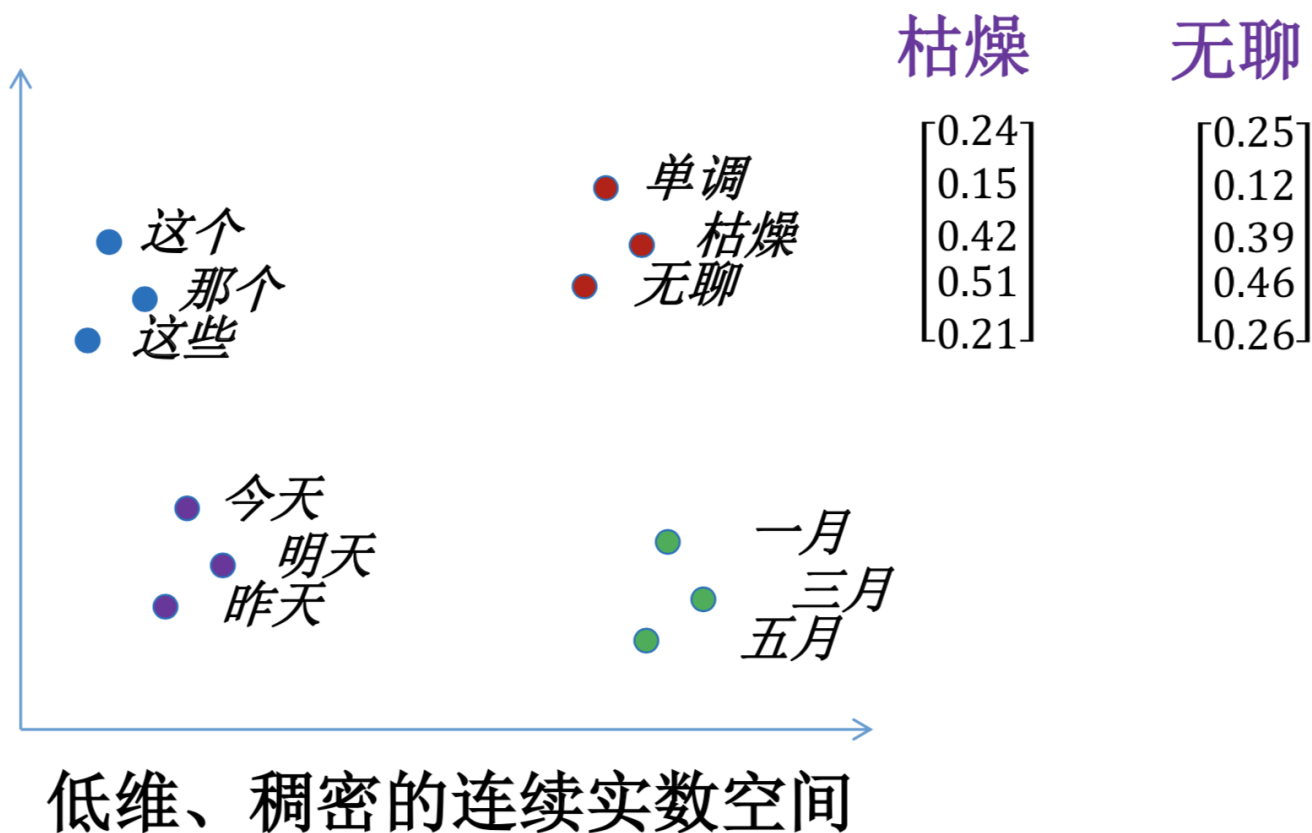


任意两个词之间的相似度都为0!



n元模型的问题

解决方法：基于连续语义空间的词语表示





n元模型的问题

解决方法：基于连续语义空间的词语表示

很

$$\begin{bmatrix} 0.01 \\ 0.59 \\ 0.18 \\ 0.05 \\ 0.47 \end{bmatrix}$$

$P(\text{无聊} |$

$$P \left(\begin{bmatrix} 0.25 \\ 0.12 \\ 0.39 \\ 0.46 \\ 0.26 \end{bmatrix} \middle| \begin{bmatrix} 0.01 \\ 0.59 \\ 0.18 \\ 0.05 \\ 0.47 \end{bmatrix} \right)$$

vs.

$P(\text{枯燥} |$

$$P \left(\begin{bmatrix} 0.24 \\ 0.15 \\ 0.42 \\ 0.51 \\ 0.21 \end{bmatrix} \middle| \begin{bmatrix} 0.01 \\ 0.59 \\ 0.18 \\ 0.05 \\ 0.47 \end{bmatrix} \right)$$

$$f \begin{pmatrix} 0.01 \\ 0.59 \\ 0.18 \\ 0.05 \\ 0.47 \\ 0.25 \\ 0.12 \\ 0.39 \\ 0.46 \\ 0.26 \end{pmatrix}$$

vs.

$$f \begin{pmatrix} 0.01 \\ 0.59 \\ 0.18 \\ 0.05 \\ 0.47 \\ 0.24 \\ 0.15 \\ 0.42 \\ 0.51 \\ 0.21 \end{pmatrix}$$



提纲

- ① 基本概念
- ② n 元文法模型
- ③ n 元模型应用
- ④ n 元模型的问题
- ⑤ 语言模型的发展

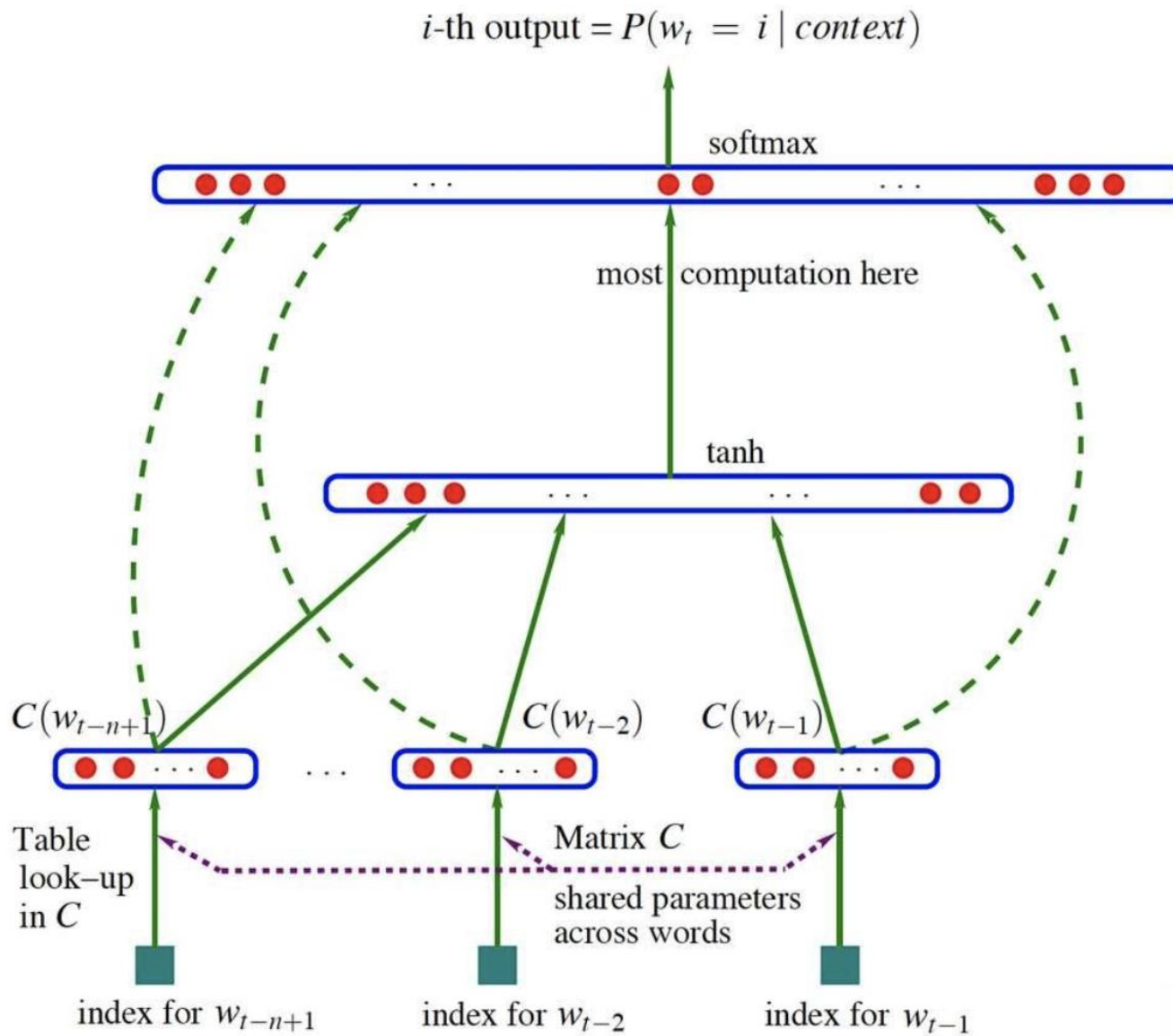


语言模型的发展

- 阶段一（约1970之前），基于**规则**的语言模型：主要基于手写规则。
- 阶段二（约1970-约2000），基于**统计**的语言模型：从数学统计的角度预测下个词的出现概率，如N-Gram等，推理过程非常直观，但是推理结果非常受数据集的影响，容易出现数据稀疏（即空值）等问题。
- 阶段三（约2000-约2017），基于**神经网络**的语言模型：比如NNLM、RNN、LSTM等。
- 阶段四（约2018-现在），基于Transformer的语言模型：比如BERT、GPT系列等大模型。



NNLM





RNN

output distribution

$$y_4 = \text{softmax}(Uh_4 + b_2) \in \mathbb{R}^{|V|}$$

hidden states

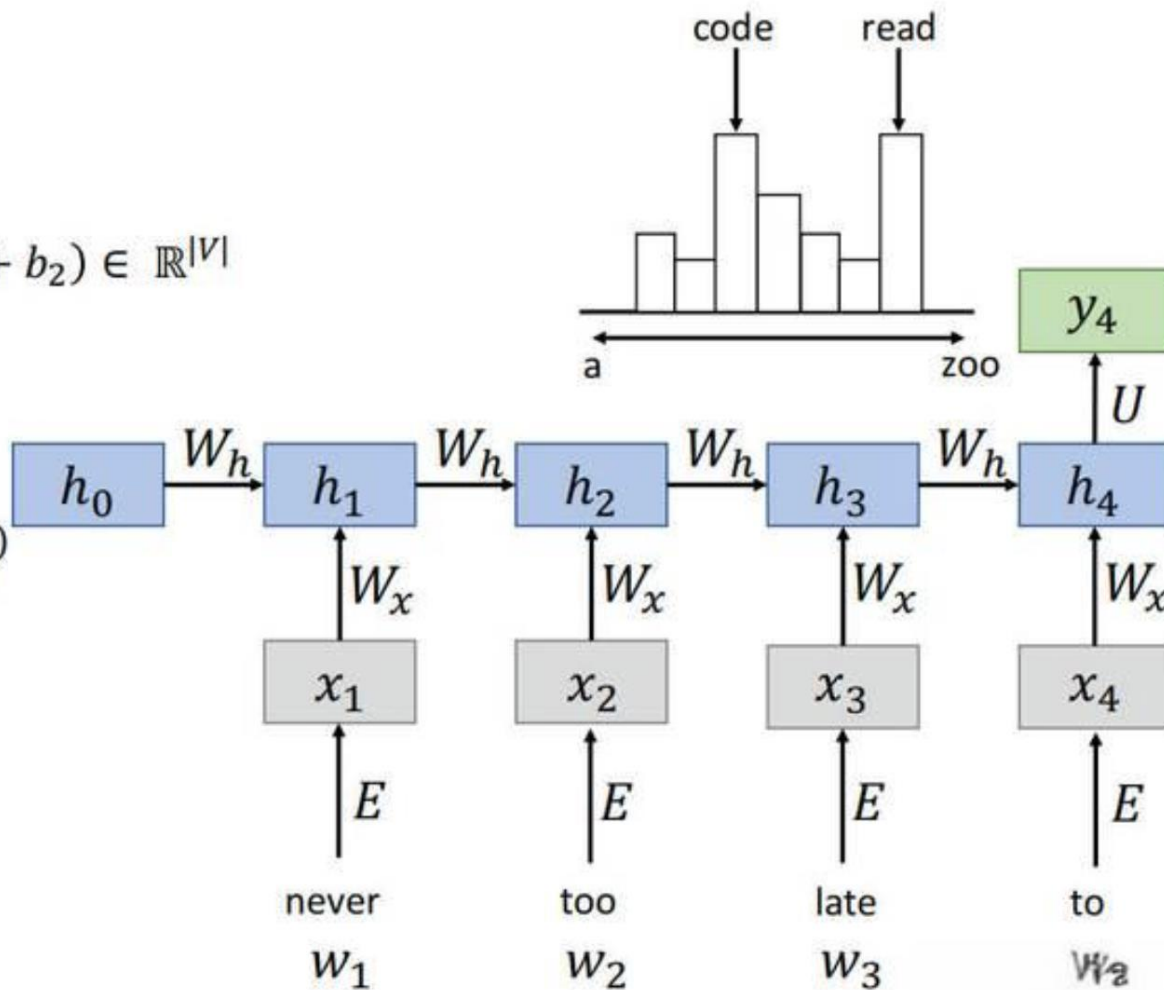
$$h_i = \tanh\left(\frac{W_x x_i + W_h h_{i-1} + b_1}{2}\right)$$

word embeddings

$$x_i = Ew_i$$

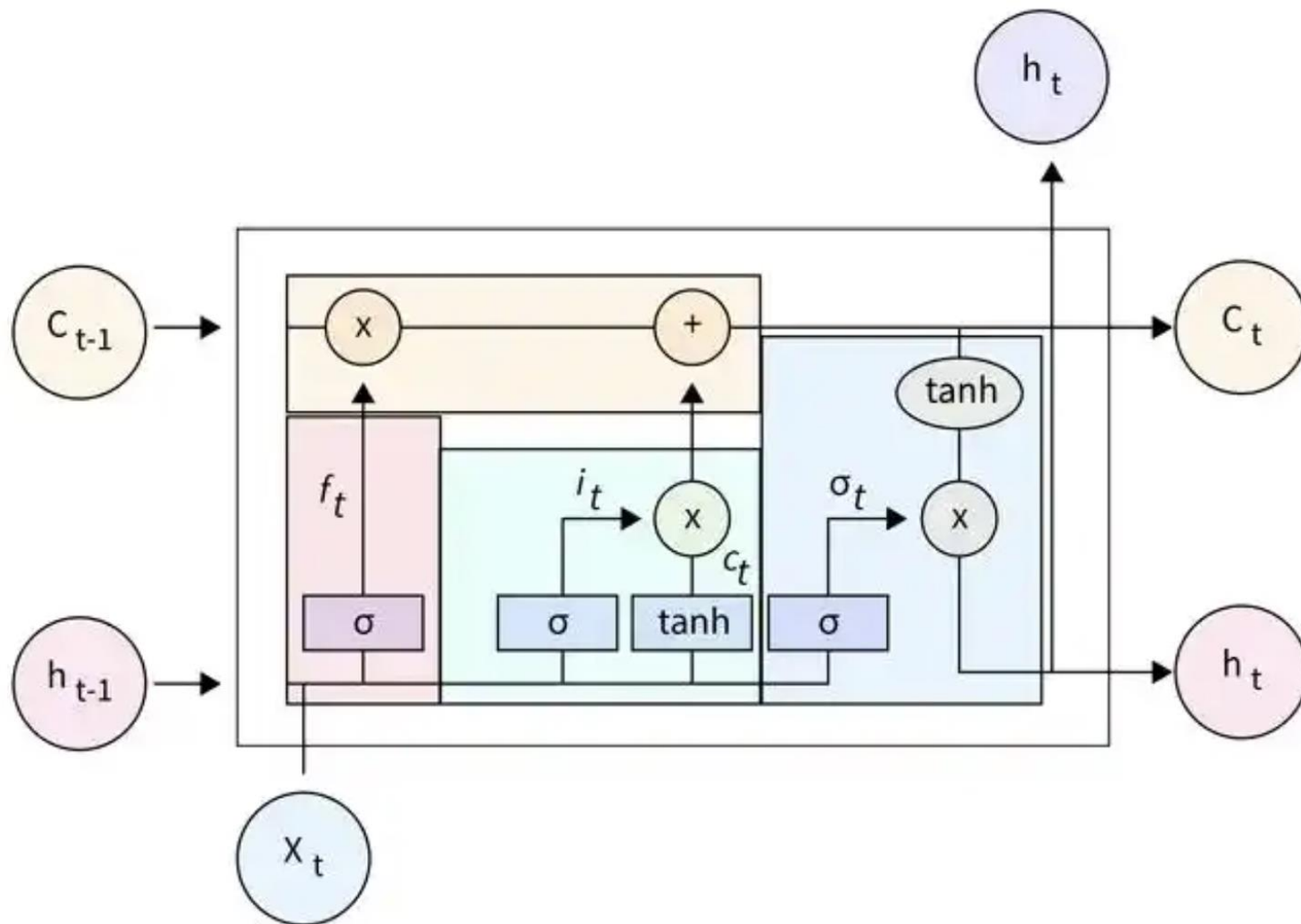
one-hot vectors

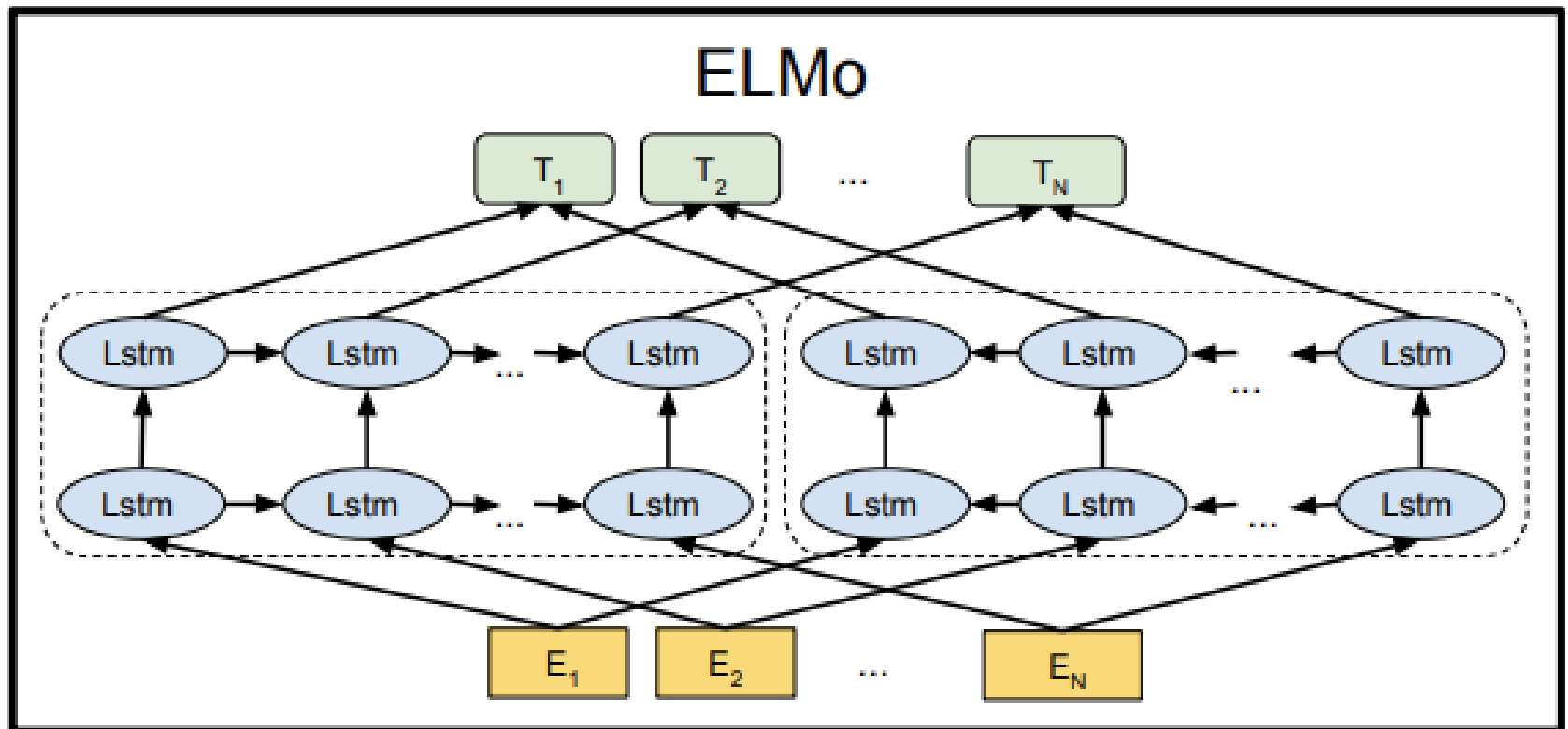
$$w_i \in \mathbb{R}^{|V|}$$





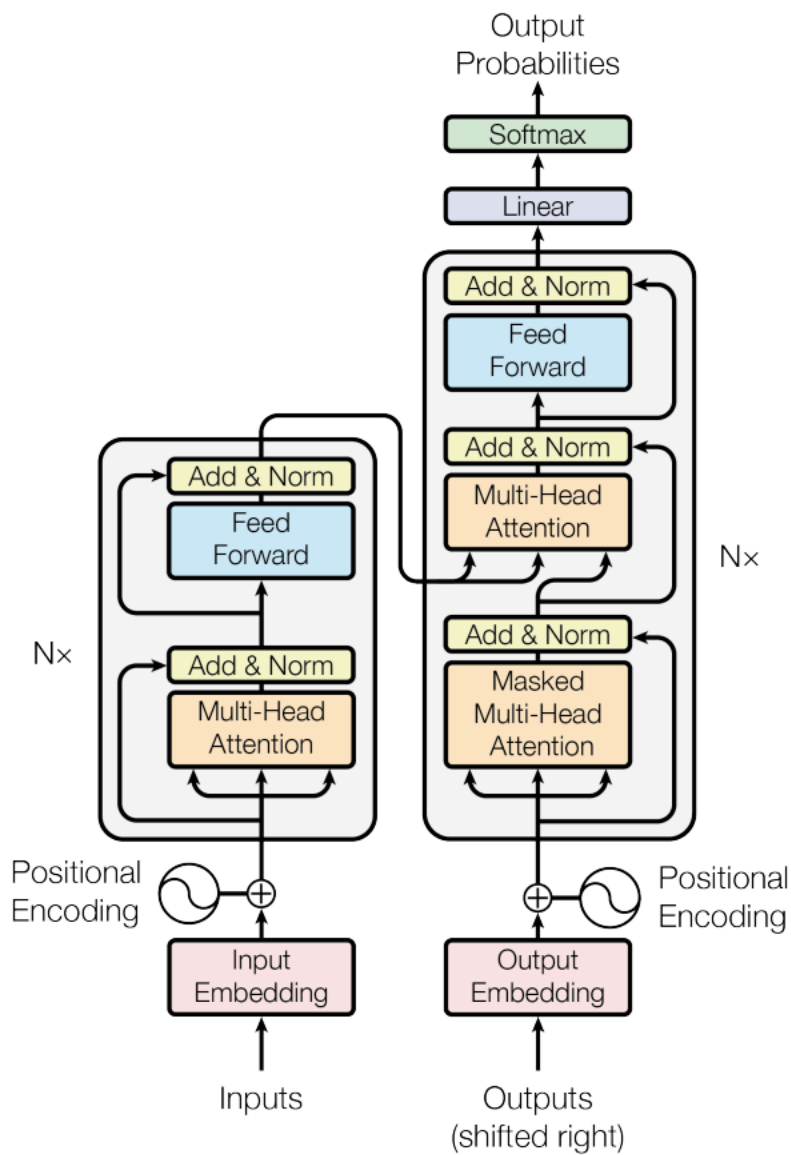
LSTM

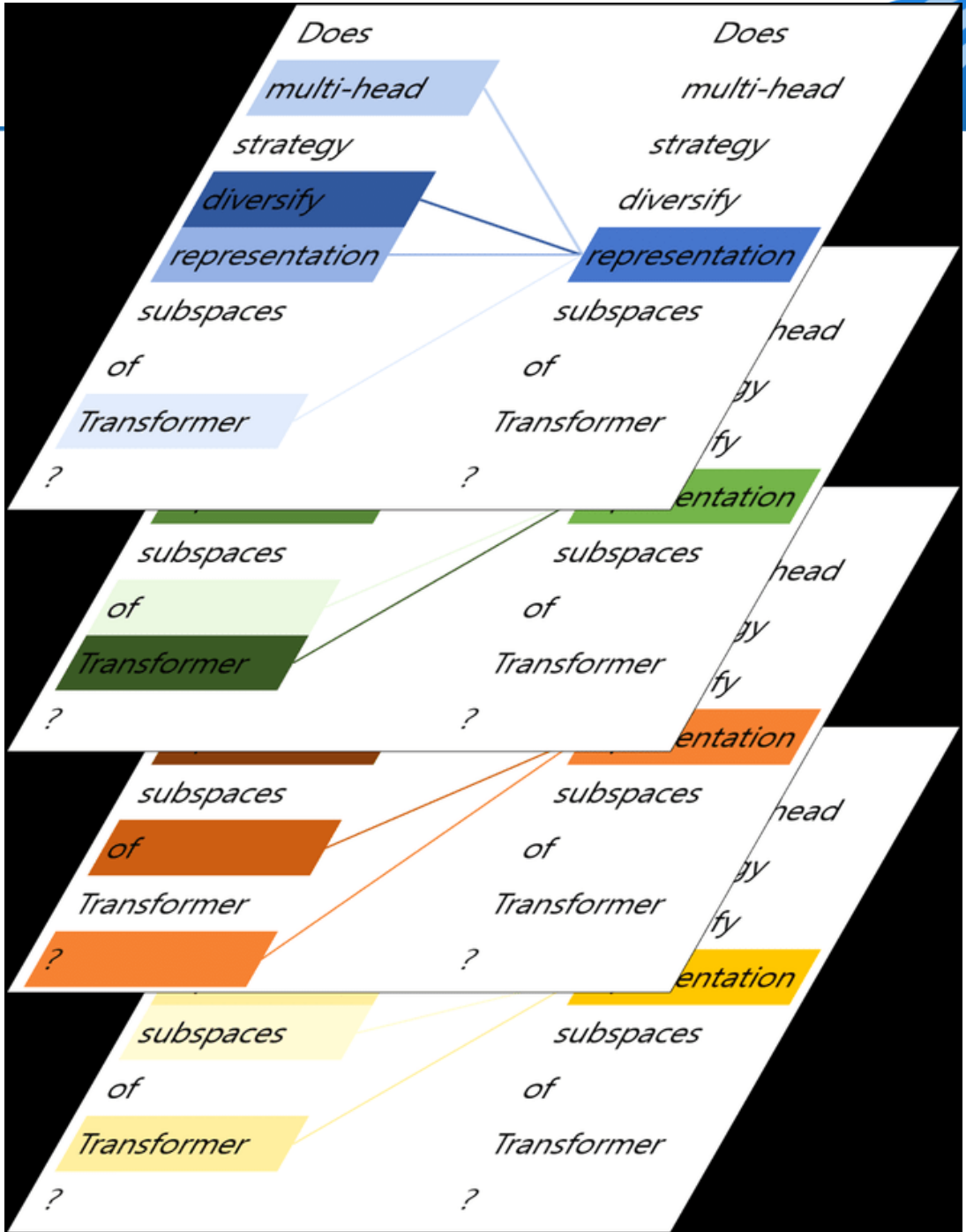






Transformer

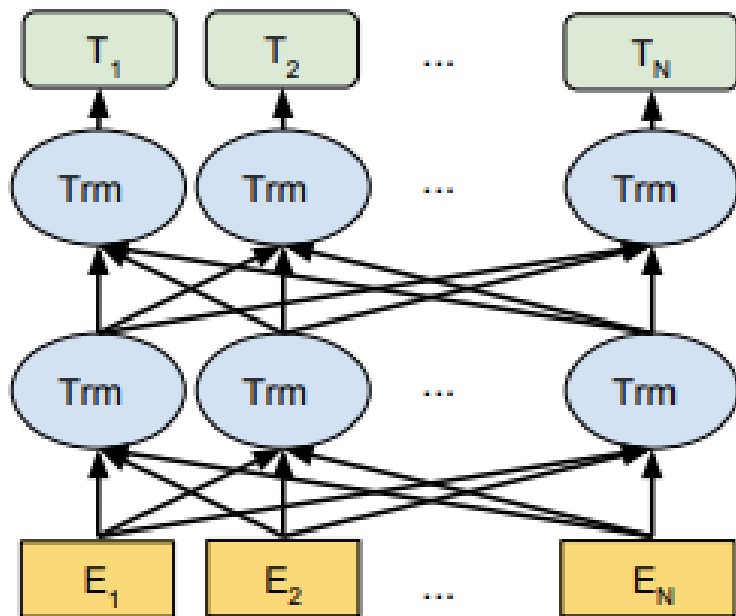




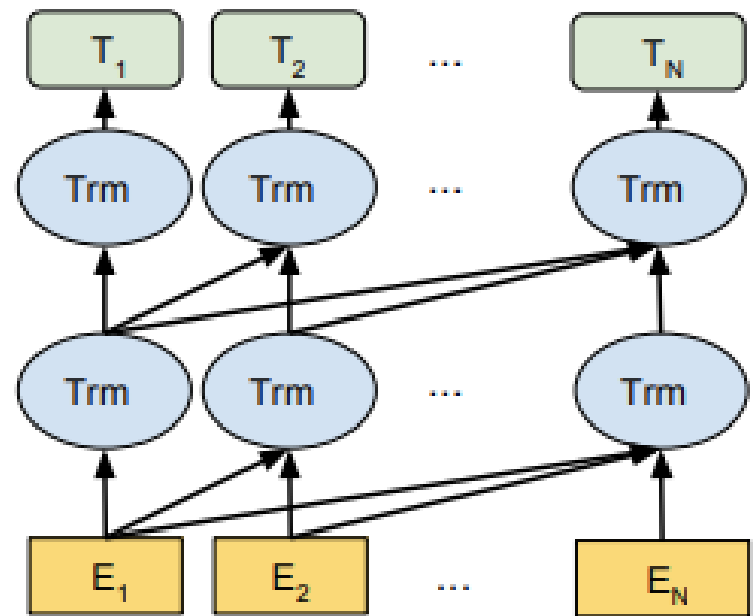


BERT、GPT

BERT (Ours)



OpenAI GPT





- 宗成庆:《自然语言处理》
- CBOW、Skip-Gram.....
- Word2Vec、GloVe、FastText.....
- RNN、LSTM、Transformer.....
- ELMo、BERT、GPT.....



北京航空航天大学计算机学院

School of Computer Science and Engineering, Beihang University

谢谢

